

[GRENOBLE INP - ESISAR]

PX222 - AUTO ELEC

Carnet de Bord

Contrôle de courant dans un circuit inductif

Antton Chevrier Lucas Fernandez Francis Ferguson

Période : Mars à juin 2026

Dernière mise à jour : 4 juin 2026

Table des matières

1	Mercredi 18 mars 2026	3
1.1	Modélisation des éléments du montage	3
1.1.1	Bobine	3
1.1.2	Amplificateur	3
1.1.3	Capteur de courant	4
1.1.4	Calcul de la boucle ouverte	4
1.2	Matériel et Sécurité	4
1.2.1	Liste du matériel prévu	4
1.2.2	Consignes de sécurité et protocoles	4
1.3	Pour la prochaine séance	5
2	Mardi 31 mars 2026	7
2.1	Analyse du signal de commande	9
2.2	Conclusion de séance	9
	Interséance	10
3	Vendredi 24 avril 2026	13
3.1	Dimensionnement du correcteur analogique	13
3.1.1	Étage Proportionnel ($U205A$)	13
3.1.2	Étage Intégral ($U205B$)	13
3.1.3	Tableau récapitulatif des composants	13
3.1.4	Configuration des cavaliers (Jumpers)	14
3.2	Difficultés rencontrées et perspectives	14
3.3	Analyse des résultats Simulink	15
4	Mardi 05 mai 2026	16
4.1	Étude de robustesse : Influence de la variation de l'inductance L	16
4.1.1	Observations expérimentales	16
4.1.2	Conclusion sur la robustesse	17
4.2	Optimisation de la commande : Gestion de la saturation	17
4.2.1	Principe de la déconnexion de l'intégrateur	17
4.3	Mise à jour de l'algorithme de commande	17
4.3.1	Conclusion de la séance 4	18
5	Semaine de projet	19
5.1	Adaptation matérielle du correcteur	19
5.2	Analyse des relevés expérimentaux et performances	19
5.2.1	Régime transitoire et respect du cahier des charges	19
5.2.2	Régime permanent et erreur statique	20
5.2.3	Passage de 40 à 60V	21
5.2.4	Bilan de l'implémentation du correcteur numérique et limites de l'approche	23

Introduction

L'objectif de ce projet est de réaliser un asservissement de courant dans un circuit inductif, au travers de la bobine étudiée en PX221. Nous utiliserons un microcontrôleur STM-32 pour réaliser l'asservissement numérique.

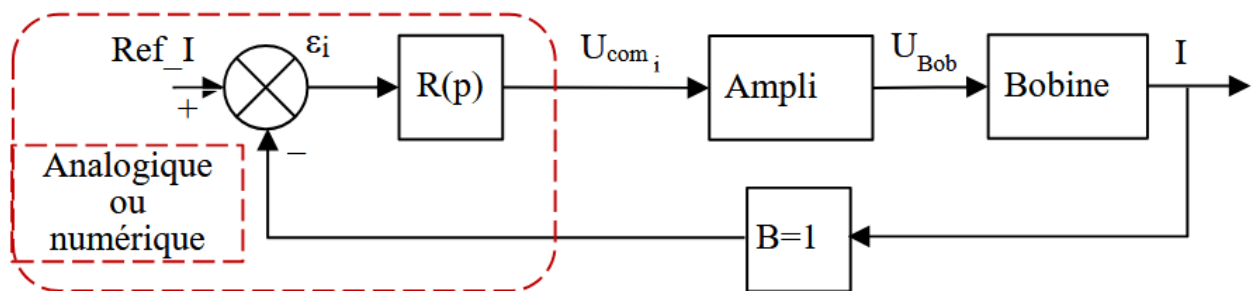


FIGURE 1 – Schéma bloc du montage à réaliser

Séance 1

Mercredi 18 mars 2026

1.1 Modélisation des éléments du montage

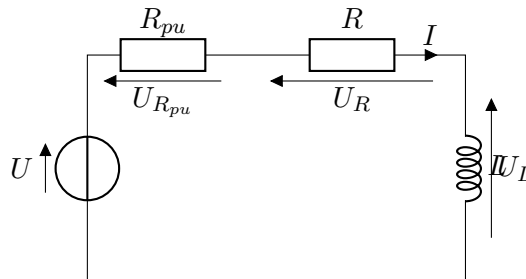
Pour commencer, nous avons modélisé les différents éléments du montage à l'aide de leurs fonctions de transfert respectives.

1.1.1 Bobine

Pour l'étude, nous retenons un **modèle série simple** de la bobine à noyau de fer (BNF). Ce modèle prend en compte l'inductance L représentant l'auto-induction et la résistance série R constituée par l'enroulement en cuivre.

Nous y ajoutons en série une résistance de puissance $R_{pu} = 12\ \Omega$ afin de diviser la constante de temps du système par 2. D'après les mesures du semestre précédent (PX221), nous savons que la résistance interne de la bobine est $R = 14\ \Omega$ et son inductance est $L = 1,2\ \text{H}$.

La résistance totale du circuit est donc : $R_{tot} = R + R_{pu} = 14 + 12 = 26\ \Omega$.



On obtient la fonction de transfert de la charge :

$$H_B(p) = \frac{I}{U} = \frac{H_0}{1 + \tau p} \quad \text{avec } H_0 = \frac{1}{R_{tot}} = \frac{1}{26} \text{ et } \tau = \frac{L}{R_{tot}} = \frac{1,2}{26} \approx 0,046 \text{ s}$$

1.1.2 Amplificateur

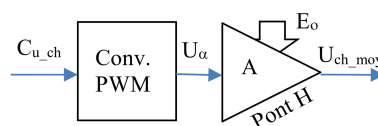


FIGURE 1.1 – Schéma bloc fonctionnel de l'amplificateur

L'amplificateur est un dispositif à découpage avec une structure en pont en H. Nous utilisons un **modèle moyen** pour l'asservissement. En effet, la fréquence de découpage ($f > 20\ \text{kHz}$) est fixée pour

être inaudible et est très supérieure à la constante de temps de la charge. Cela permet d'approximer la tension aux bornes de la bobine par sa valeur moyenne $U_{ch_{moy}}$. D'après le sujet, la tension de sortie est donnée par : $U_{ch_{moy}} = (2U_{com} - 1)E_0$. Le gain statique de l'amplificateur est alors défini par la dérivée :

$$K_A = \frac{dU_{Bob}}{dU_{com}} = 2E_0$$

1.1.3 Capteur de courant

La mesure du courant s'effectue via la résistance de puissance R_{pu} . Le signal traverse un diviseur de tension par 12 pour rester dans la plage de tension de l'électronique, puis un amplificateur d'instrumentation ($U201$). Ce montage permet d'obtenir une conversion directe au point de test $TP201$: $1\text{ V} \equiv 1\text{ A}$. On en déduit le gain de la chaîne de retour : $B = 1\text{ V/A}$.

1.1.4 Calcul de la boucle ouverte

La fonction de transfert en boucle ouverte (FTBO) du système complet est :

$$\begin{aligned} H_{BO}(p) &= R(p) \cdot K_A \cdot H_B(p) \cdot B \\ &= k \cdot (2E_0) \cdot \frac{1}{1 + \tau p} \cdot 1 \\ &= \boxed{\frac{2kE_0}{1 + \tau p}} \end{aligned}$$

1.2 Matériel et Sécurité

1.2.1 Liste du matériel prévu

Pour la mise en œuvre pratique de cet asservissement, nous utiliserons les équipements suivants :

- **Maquette PX222** : Elle servira de base pour le pont en H, les drivers de MOSFET et le conditionnement des signaux.
- **Bobine à Noyau de Fer (BNF)** : Identifiée en PX221, elle constituera notre charge inductive principale ($L \approx 1,2\text{ H}$).
- **Résistance de puissance ($12\ \Omega$)** : Elle sera placée en série pour réduire la constante de temps et servira également au capteur de courant.
- **Microcontrôleur STM32 (Nucleo-F103RB)** : Il sera utilisé pour l'acquisition des signaux (ADC) et la génération de la commande PWM numérique.
- **Appareillage de mesure** : Nous aurons besoin d'un oscilloscope numérique, d'un GBF pour la consigne de courant et de multimètres.
- **Alimentation de puissance** : Une source de tension continue réglable sera utilisée pour alimenter le pont en H (maximum 60 V).

1.2.2 Consignes de sécurité et protocoles

La manipulation de puissances importantes imposera le respect strict des consignes suivantes :

- **Protection électrique** : Toute tension supérieure à 50 V présentant un danger mortel, nous serons donc extrêmement attentifs lors des essais sous tension.
- **Sondes différentielles** : Pour éviter tout court-circuit via la terre de l'oscilloscope lors des mesures sur le réseau ou le pont flottant, l'usage de sondes différentielles sera obligatoire.
- **Risque thermique** : La résistance de puissance pourra atteindre des températures supérieures à 100°C ; nous éviterons tout contact direct durant et après les essais.

1.3 Pour la prochaine séance

Nous devons avoir terminé les calculs des fonctions de transfert en boucle ouvertes et fermée, ainsi que le calcul du correcteur proportionnel.

Interséance

Nous avons calculé la fonction de transfert en boucle fermée du système.

$$\begin{aligned} H_{BF}(p) &= \frac{H_{BO}(p)}{1 + H_{BO}(p)} \\ &= \frac{\frac{2kE_0}{R+R_{pu}} \frac{1}{1+\tau p}}{1 + \frac{2kE_0}{R+R_{pu}} \frac{1}{1+\tau p}} \\ &= \frac{K_{BO}}{1 + \tau p + K_{BO}} \quad \text{avec } K_{BO} = \frac{2kE_0}{R + R_{pu}} \\ &= \frac{\frac{K_{BO}}{1+K_{BO}}}{1 + \frac{\tau p}{1+K_{BO}}} \end{aligned}$$

On souhaite également une erreur statique de 5%. Or

$$\varepsilon = \frac{1}{1 + K_{BO}} = \frac{1}{1 + \frac{2kE_0}{R+R_{pu}}} = 0.05$$

donc

$$1 + \frac{2kE_0}{R + R_{pu}} = \frac{1}{0.05}$$

d'où

$$k = \left(\frac{1}{0.05} - 1 \right) \frac{R + R_{pu}}{2E_0}$$

En faisant l'application numérique avec $R + R_{pu} = 26 \, \Omega$ et $E_0 = 60 \, \text{V}$, on trouve $k \approx 4.1$

Séance 2

Mardi 31 mars 2026

Bien que théoriquement, le gain proportionnel de 4 calculé précédemment fonctionne en théorie, il ne sera pas réalisable en pratique. En effet, le signal de commande U_{com} issu du correcteur doit piloter un convertisseur PWM dont la plage d'entrée est strictement limitée entre 0 V et 1 V.

Pour une consigne unitaire de 1 A, un gain $k = 4,1$ entraînerait une commande $U_{com} = 4,1$ V dès le début de l'échelon, provoquant une saturation immédiate du rapport cyclique à 100 %. Cette saturation rendrait le système non-linéaire et invaliderait nos modèles de transfert.

Nous fixons donc $k = 1$ afin de garantir que, même pour une erreur maximale ($\varepsilon = 1$), la commande reste dans la plage linéaire (1 V). Pour compenser l'erreur statique tout en respectant cette contrainte, nous étudions un correcteur Proportionnel-Intégral (PI).

Pour le réglage du paramètre d'intégration ω_i , nous utilisons la méthode de **compensation de pôle**. L'objectif est d'annuler le pôle lent de la bobine ($\tau \approx 0,046$ s) en réglant la constante de temps du correcteur telle que $T_i = 1/\omega_i = \tau$.

Cela donne une valeur théorique de départ :

$$\omega_i = \frac{1}{\tau} = \frac{R_{tot}}{L} = \frac{26}{1,2} \approx 21,67 \text{ rad/s}$$

Nous utilisons pour cela un script matlab qui nous permet de faire varier ω_i , avec un gain $k = 1$ et de visualiser la réponse temporelle du système.

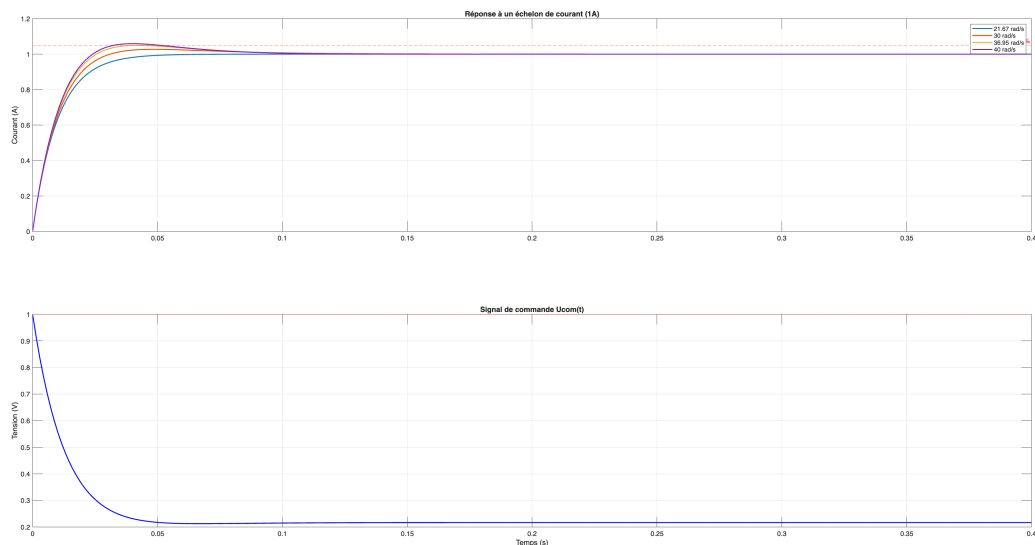


FIGURE 2.1 – Réponse temporelle du système pour différentes valeur de ω_i


```

1  % --- Parametres du systeme ---
2  Rs = 14;
3  Rext = 12;
4  Rtot = Rs + Rext;
5  L = 1.2;
6  tau = L / Rtot;
7  E0 = 60;
8  Ka = 2 * E0;
9  B = 1;
10
11 s = tf('s');
12 H_charge = (1/Rtot) / (1 + tau*s);
13 H = Ka * H_charge * B;
14
15 % --- Optimisation du PI ---
16 k = 1; % Gain fixe pour eviter la saturation
17 wi_tests = [21.67, 30, 36.95, 40]; % Inclut la compensation de pole
18
19 figure('Name', 'Reponse_PI_et_Commande');
20 for wi = wi_tests
21     R = k * (1 + wi/s);
22     FTBO = R * H;
23     FTBF = feedback(FTBO, 1);
24
25     % Analyse des performances
26     info = stepinfo(FTBF);
27     [y, t] = step(FTBF, 0.4);
28
29     subplot(2,1,1); plot(t, y, 'LineWidth', 1.5); hold on;
30     fprintf('wi=%0.2f | Depassement=%0.2f%% | Tr(5%%)=%0.3f s\n', ...
31             wi, info.Overshoot, info.SettlingTime);
32 end
33
34 % --- Mise en page graphique ---
35 subplot(2,1,1);
36 yline(1.05, 'r--', 'Limite_5%'); grid on;
37 title('Reponse_a_un_echelon_de_courant(1A)');
38 ylabel('Courant(A)');
39 legend(string(wi_tests) + " rad/s");
40
41 % Visualisation de la commande pour wi choisi
42 R_opt = k * (1 + 36.95/s);
43 Ucom = feedback(R_opt, H); % Transfert Consigne -> Commande
44 [u, t_u] = step(Ucom, 0.4);
45 subplot(2,1,2); plot(t_u, u, 'b', 'LineWidth', 1.5); hold on;
46 yline(1, 'r--', 'Saturation_PWM(1V)'); grid on;
47 title('Signal_de_commande_Ucom(t)');
48 ylabel('Tension(V)'); xlabel('Temps(s)');

```

Listing 2.1 – Script Matlab pour trouver le bon paramètre ω_i

L'analyse des courbes montre que la valeur $\omega_i = 36,95 \text{ rad/s}$ offre le meilleur compromis. Bien qu'elle soit supérieure à la valeur théorique de compensation, elle permet d'accélérer le système tout en restant sous la limite des 5 % de dépassement.

Les performances simulées sont les suivantes :

- **Dépassement** : 4,92 % (inférieur à la limite de 5 %).
- **Temps de réponse (5 %)** : environ 65 ms.
- **Erreur statique** : nulle (grâce à l'action intégrale).

2.1 Analyse du signal de commande

Le second graphique de la Figure 2.1 présente l'évolution de la tension de commande $U_{com}(t)$. Cette analyse est cruciale pour valider la faisabilité technologique de notre asservissement :

- **Respect de la plage linéaire** : On observe que pour le réglage optimal ($\omega_i = 36,95 \text{ rad/s}$), le signal débute à exactement 1 V sans jamais le dépasser. Cela confirme que notre choix $k = 1$ permet d'exploiter la dynamique maximale du convertisseur PWM (limité à 1 V d'après le sujet) sans jamais saturer le rapport cyclique à 100 %.
- **Effort de commande** : Le pic initial à 1 V illustre l'action proportionnelle qui fournit l'énergie nécessaire pour vaincre l'inertie de la charge inductive. La décroissance qui suit montre l'action du terme intégral qui ajuste finement la tension pour stabiliser le courant à 1 A, prouvant que le système ne force pas inutilement sur les composants une fois le régime établi.

2.2 Conclusion de séance

Pour la prochaine séance, nous devons nous renseigner sur l'utilisation et la programmation de carte STM32, notamment sur les différents ports disponibles les différentes façon de brancher les différents élément du montage à la carte.

Interséance

Suite aux simulations effectuées lors de la séance précédente, nous avons préparé l'implémentation du correcteur PI sur le microcontrôleur STM32 (Nucleo-F103R8) afin de réaliser l'asservissement numérique.

Stratégie de programmation

Pour garantir la précision temporelle de l'asservissement, nous avons choisi une programmation par accès direct aux registres. Les choix techniques sont les suivants :

- **PWM (TIM1_CH1 sur PA8)** : Configuration d'une porteuse à environ 22 kHz pour piloter le pont en H de la maquette.
- **Acquisition (ADC1_IN5 sur PA5)** : Lecture de l'image du courant convertie à 1 V/1 A par l'amplificateur d'instrumentation de la carte.
- **Régulation** : Implémentation de la loi de commande avec $k = 1$ et $\omega_i = 36,95 \text{ rad/s}$. Une saturation logicielle à 1 V est ajoutée pour protéger le système.

Code source concernant les registres et le setup de la carte

```
1  #include <stdint.h>
2
3  // --- ADRESSES REGISTRES STM32F103 ---
4  #define RCC_BASE      0x40021000
5  #define GPIOA_BASE    0x40010800
6  #define ADC1_BASE     0x40012400
7  #define TIM1_BASE     0x40012C00
8
9  // Horloges
10 #define RCC_APB2ENR    (*(volatile uint32_t *) (RCC_BASE + 0x18))
11 #define RCC_CFGR       (*(volatile uint32_t *) (RCC_BASE + 0x04))
12
13 // GPIOA
14 #define GPIOA_CRL      (*(volatile uint32_t *) (GPIOA_BASE + 0x00))
15 #define GPIOA_CRH      (*(volatile uint32_t *) (GPIOA_BASE + 0x04))
16
17 // ADC1 (PA5 est ADC_IN5 sur F103)
18 #define ADC1_SR        (*(volatile uint32_t *) (ADC1_BASE + 0x00))
19 #define ADC1_CR2       (*(volatile uint32_t *) (ADC1_BASE + 0x08))
20 #define ADC1_SQR3      (*(volatile uint32_t *) (ADC1_BASE + 0x34))
21 #define ADC1_DR        (*(volatile uint32_t *) (ADC1_BASE + 0x4C))
22
23 // TIM1 (PWM sur PA8)
24 #define TIM1_CR1       (*(volatile uint32_t *) (TIM1_BASE + 0x00))
25 #define TIM1_PSC       (*(volatile uint32_t *) (TIM1_BASE + 0x28))
26 #define TIM1_ARR       (*(volatile uint32_t *) (TIM1_BASE + 0x2C))
27 #define TIM1_CCR1      (*(volatile uint32_t *) (TIM1_BASE + 0x34))
28 #define TIM1_CCMR1     (*(volatile uint32_t *) (TIM1_BASE + 0x18))
```

```

29 #define TIM1_CCER      (*(volatile uint32_t *) (TIM1_BASE + 0x20))
30 #define TIM1_BDTR      (*(volatile uint32_t *) (TIM1_BASE + 0x44))
31
32 void setup(void) {
33     // 1. Activer horloges : GPIOA, ADC1, TIM1 et AFIO
34     RCC_APB2ENR |= (1 << 0) | (1 << 2) | (1 << 9) | (1 << 11);
35
36     // 2. Configurer PA8 en "Alternate Function Push-Pull" (PWM TIM1_CH1)
37     GPIOA_CRH &= ~(0xF << 0);
38     GPIOA_CRH |= (0xB << 0); // Speed 50MHz, AF Push-Pull
39
40     // 3. Configurer PA5 en mode Analogique (Entree mesure courant)
41     GPIOA_CRL &= ~(0xF << 20);
42
43     // 4. Configurer TIM1 pour PWM a 22kHz
44     TIM1_PSC = 0; // Pas de diviseur
45     TIM1_ARR = 3272; // 72MHz / 22kHz environ = 3272
46     TIM1_CCMR1 |= (0x6 << 4); // Mode PWM 1 sur CH1
47     TIM1_CCER |= (1 << 0); // Activer sortie CH1
48     TIM1_BDTR |= (1 << 15); // Main Output Enable (MOE) - Specifique
49     TIM1 TIM1 // Lancer Timer
50     TIM1_CR1 |= (1 << 0);
51
52     // 5. Configurer ADC1
53     ADC1_SQR3 = 5; // Selectionner canal 5 (PA5)
54     ADC1_CR2 |= (1 << 0); // Allumer ADC
55 }
56
57 uint32_t read_ADC(void) {
58     ADC1_CR2 |= (1 << 22); // Lancer conversion
59     while (!(ADC1_SR & (1 << 1))); // Attendre fin
60     return ADC1_DR;
61 }
62
63 void delay_ms(uint32_t ms) {
64     for (uint32_t i = 0; i < ms * 8000; i++) __asm__("nop");
65 }

```

Listing 2.2 – Implémentation du correcteur PI sur STM32F103

Code source concernant les calculs du correcteur PI et de la commande

```

1 // --- PARAMETRES DU CORRECTEUR PI ---
2 const float Kp = 1.0f; // Gain proportionnel
3 const float Wi = 36.95f; // Parametre d'integration
4 const float Ts = 0.001f; // Periode d'echantillonnage (1ms)
5 float integrale = 0.0f;
6 float consigne = 1.0f; // 1V ==> 1A
7
8 while (1) {
9     // --- 1. ACQUISITION ---
10    uint32_t val_adc = read_ADC();
11    // Conversion ADC (12 bits) en Volts : val * 3.3 / 4095
12    float courant_mesure = (float)val_adc * (3.3f / 4095.0f);
13
14    // --- 2. CALCUL DU PI ---
15    float erreur = consigne - courant_mesure;
16    integrale += erreur * Ts;
17

```

```

18 // Commande = Kp * (erreur + Wi * integrale)
19 float commande = Kp * (erreur + Wi * integrale);
20
21 // --- 3. SATURATION ET PROTECTION ---
22 // On bride la commande entre 0 et 1V pour rester en zone lineaire
23 if (commande > 1.0f) commande = 1.0f;
24 if (commande < 0.0f) commande = 0.0f;
25
26 // --- 4. ACTION (PWM) ---
27 // Le rapport cyclique est proportionnel a la commande (0V=0, 1V
    =100%)
28 // On ramene la commande 0-1V vers l'ARR du Timer (3272)
29 TIM1_CCR1 = (uint32_t)(commande * 3272.0f);
30
31 delay_ms(1); // Echantillonnage a 1kHz
32 }

```

Listing 2.3 – Implémentation du correcteur PI sur STM32F103

Séance 3

Vendredi 24 avril 2026

3.1 Dimensionnement du correcteur analogique

Pour transposer nos résultats de simulation sur la maquette PX222, nous devons déterminer les valeurs des composants (résistances et condensateurs) à installer sur les supports tulipes des amplificateurs opérationnels $U205A$ et $U205B$.

3.1.1 Étage Proportionnel ($U205A$)

Le premier étage réalise le gain proportionnel k via une structure d'amplificateur inverseur. Le gain est défini par le rapport des résistances R_{211} et R_{208} :

$$k = \frac{R_{211}}{R_{208}}$$

Afin de respecter notre réglage $k = 1$ (pour éviter la saturation du PWM), nous choisissons des résistances identiques :

— **Choix** : $R_{208} = 10\text{ k}\Omega$ et $R_{211} = 10\text{ k}\Omega$.

3.1.2 Étage Intégral ($U205B$)

Le second étage réalise l'action intégrale $(1 + \frac{\omega_i}{p})$ via une structure non-inverseuse. Le paramètre d'intégration ω_i est lié aux composants R_{212} et C_{211} par la relation :

$$\omega_i = \frac{1}{R_{212} \cdot C_{211}}$$

D'après nos simulations, nous avons optimisé $\omega_i = 36,95\text{ rad/s}$, ce qui impose un produit $R_{212} \cdot C_{211} \approx 0,027\text{ s}$. En utilisant les valeurs standards disponibles au magasin, nous fixons :

- **Condensateur** : $C_{211} = 1\text{ }\mu\text{F}$.
- **Résistance** : $R_{212} = \frac{1}{36,95 \cdot 10^{-6}} \approx 27\text{ k}\Omega$ (Valeur normalisée la plus proche).
- **Support** R_{215} : Lissé vide pour garantir un comportement purement intégrateur.

3.1.3 Tableau récapitulatif des composants

Composant	Référence	Valeur calculée	Fonction
Résistance	R_{208}	$10\text{ k}\Omega$	Entrée gain k
Résistance	R_{211}	$10\text{ k}\Omega$	Retour gain k
Résistance	R_{212}	$27\text{ k}\Omega$	Réglage ω_i
Condensateur	C_{211}	$1\text{ }\mu\text{F}$	Réglage ω_i

TABLE 3.1 – Valeurs des composants pour le correcteur analogique

3.1.4 Configuration des cavaliers (Jumpers)

Le paramétrage de la maquette PX222 s'effectue via une série de cavaliers. Le tableau suivant récapitule les positions nécessaires pour basculer entre les modes analogique et numérique, ainsi que les fonctions de sécurité :

Cavalier	Position	Fonction / Mode de fonctionnement
J201	1-2	Fonctionnement en Boucle Fermée (Asservissement)
	2-3	Fonctionnement en Boucle Ouverte (Tests unitaires)
J202	1-2	Correction Analogique : Sélection des AOP U205
	2-3	Correction Numérique : Sélection du STM32 (PA8)
J203	OFF	Utilisation de la consigne logicielle interne
	ON	Utilisation d'une consigne externe (GBF sur PA0)
J204	ON	Connexion de la mesure de courant vers l'ADC (PA5)
J104	ON	Autorisation de la puissance vers le Pont en H

TABLE 3.2 – Synthèse de la configuration matérielle de la maquette PX222

3.2 Difficultés rencontrées et perspectives

Au cours de cette séance, nous avons tenté de simuler manuellement le comportement de la carte et de la boucle d'asservissement. Cependant, cette approche n'a pas permis d'obtenir des résultats concluants : la complexité de la gestion temporelle et de la saturation logicielle s'est avérée trop importante pour obtenir des prédictions fiables.

En conséquence, pour la séance suivante, nous prévoyons de modéliser l'intégralité du système sur **Simulink**. Ce "jumeau numérique" permettra de valider nos réglages avant les tests physiques en intégrant les blocs suivants :

- **Modélisation de la chaîne directe** : Nous utiliserons un bloc *Step* pour la consigne de 1 A, suivi de notre correcteur PI ($k = 1, \omega_i = 36,95 \text{ rad/s}$) et d'un bloc de *Saturation* bridé entre 0 V et 1 V pour simuler la limite réelle du convertisseur PWM.
- **Représentation de la puissance** : Un gain $K_A = 120$ représentera le pont en H alimenté sous 60 V, pilotant la fonction de transfert de la bobine $H_B(p) = \frac{0,038}{1+0,046p}$.
- **Objectifs de validation** : Cette simulation servira à confirmer que le dépassement reste inférieur à 5 % malgré la saturation. Nous testerons également l'impact du passage en mode discret (échantillonnage à 1 ms) pour anticiper le comportement réel sur le microcontrôleur STM32.

Cette étape de simulation globale est indispensable pour garantir que nos choix théoriques sont compatibles avec les contraintes technologiques de la maquette avant de procéder aux relevés expérimentaux finaux.

Interséance

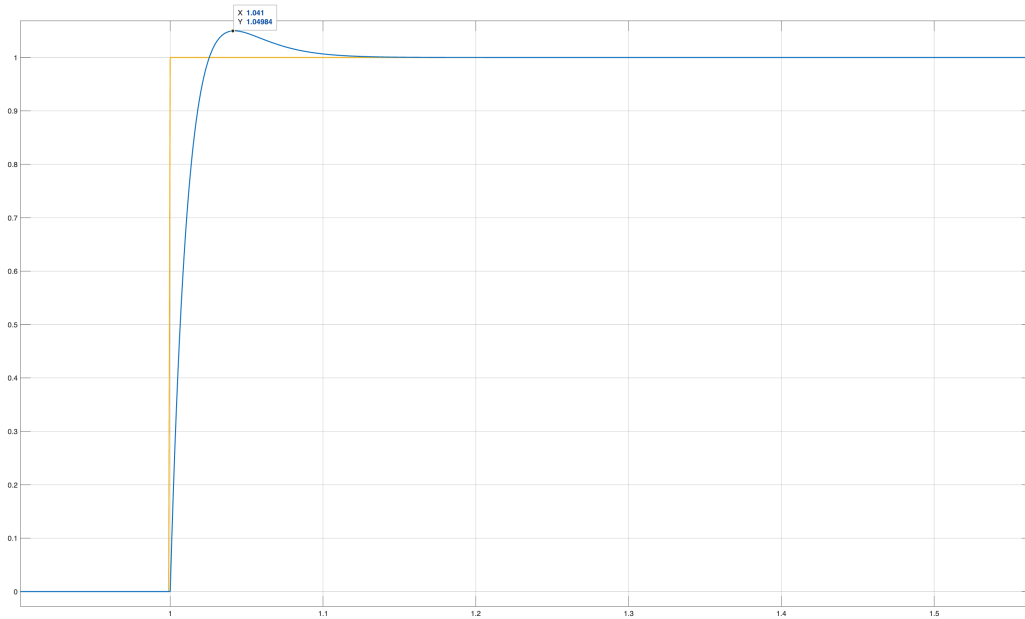


FIGURE 3.1 – Schéma de montage pour la mise en œuvre du correcteur PI sur la maquette

3.3 Analyse des résultats Simulink

La simulation effectuée avec $\omega_i = 36,95 \text{ rad/s}$ montre une réponse temporelle très satisfaisante. Nous observons :

- **Dépassement** : Environ 4,98 %. Ce léger écart par rapport aux 4,92 % théoriques s'explique par la saturation de la commande à 1 V qui sollicite davantage l'action intégrale en début d'échelon.
- **Erreur statique** : Nulle, le courant rejoint précisément la consigne de 1 A.
- **Stabilité** : Le système est parfaitement stable et le régime établi est atteint en moins de 100 ms.

Ces résultats valident notre paramétrage pour la mise en œuvre réelle sur la carte STM32 Nucléo-F103RB.

Voici le modèle simulink que nous avons utilisé pour la simulation :

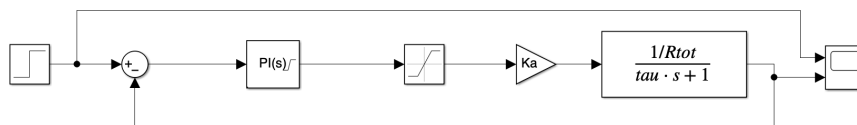


FIGURE 3.2 – Modèle Simulink de l'asservissement de courant avec correcteur PI et saturation

Séance 4

Mardi 05 mai 2026

4.1 Étude de robustesse : Influence de la variation de l'inductance L

Dans un système réel, et particulièrement avec une Bobine à Noyau de Fer (BNF), la valeur de l'inductance L peut varier en fonction du point de fonctionnement (saturation du fer) ou des imprécisions de mesure initiale. Nous avons donc simulé le comportement du correcteur PI ($k = 1, \omega_i = 36,95 \text{ rad/s}$) face à une variation de $\pm 20\%$ de la valeur nominale de L (1,2 H).

4.1.1 Observations expérimentales

Les résultats obtenus sous Simulink mettent en évidence les points suivants :

- **Augmentation de l'inductance ($L + 20\%$)** (Voir figure 4.1) : On observe un dépassement qui excède la limite des 5%, atteignant environ 7%. Cela s'explique par l'augmentation de la constante de temps $\tau = L/R_{tot}$. Le correcteur, dimensionné pour une charge plus rapide, devient "trop agressif" pour ce système ralenti, provoquant une oscillation plus marquée.
- **Diminution de l'inductance ($L - 20\%$)** (Voir figure 4.2) : À l'inverse, lorsque l'inductance diminue, le dépassement s'atténue fortement, rendant la réponse plus amortie. Le système gagne en stabilité mais perd légèrement en rapidité de montée.

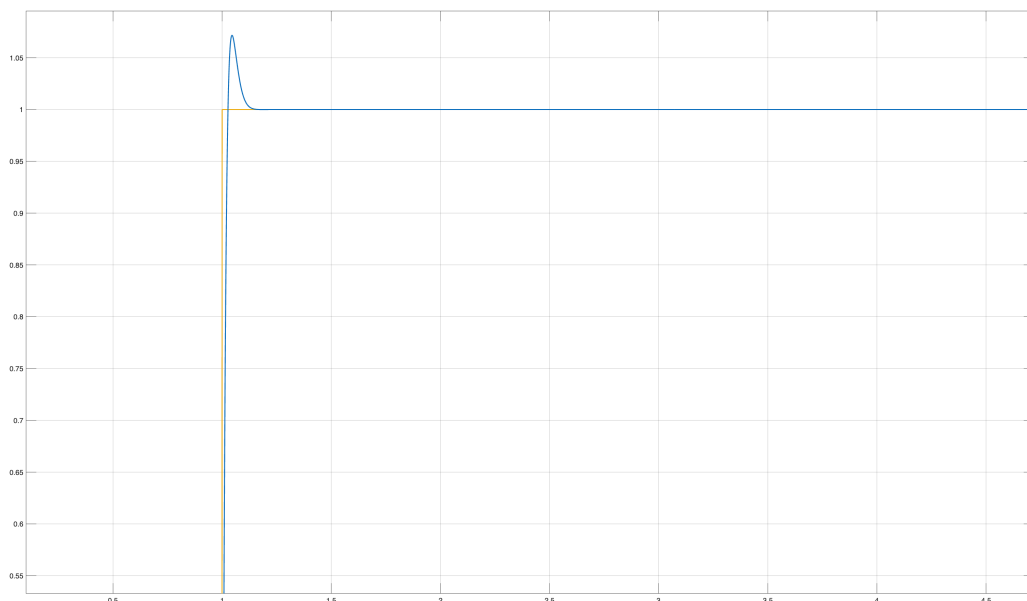


FIGURE 4.1 – Modèle Simulink de l'asservissement de courant avec correcteur PI et saturation

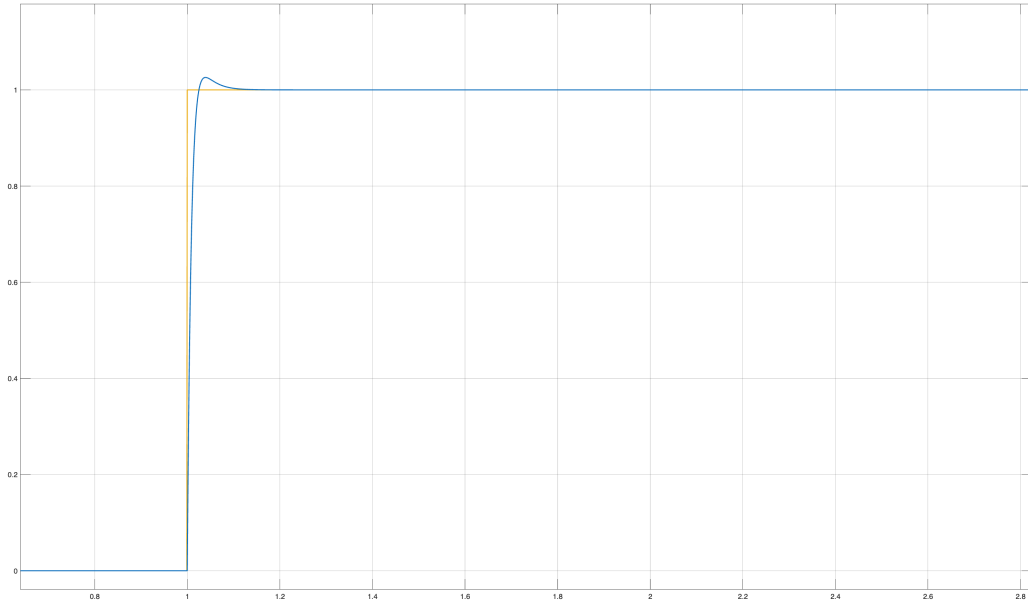


FIGURE 4.2 – Modèle Simulink de l’asservissement de courant avec correcteur PI et saturation

4.1.2 Conclusion sur la robustesse

Cette simulation démontre que notre réglage de $\omega_i = 36,95 \text{ rad/s}$ se situe à la limite de la zone de validité du cahier des charges. Une augmentation imprévue de l’inductance sur la maquette réelle pourrait nous faire dépasser les 5 % de dépassement autorisés.

4.2 Optimisation de la commande : Gestion de la saturation

L’analyse des résultats Simulink a montré que la saturation de la commande à 1 V entraînait un dépassement de 4,98 %, supérieur aux 4,92 % théoriques. Ce phénomène est dû à l’accumulation de l’erreur dans l’action intégrale pendant la phase où la commande est saturée.

4.2.1 Principe de la déconnexion de l’intégrateur

Pour remédier à ce problème, nous avons implémenté une sécurité dans le code du microcontrôleur. Le principe consiste à suspendre l’actualisation de la variable **intégrale** lorsque deux conditions sont réunies simultanément :

1. La commande calculée atteint ou dépasse la limite physique (1 V ou 0 V).
2. L’erreur est de même signe que la saturation (le système cherche à saturer davantage).

Cette modification permet d’éviter que l’intégrateur ne "s’emballe" durant la phase transitoire. Dès que l’erreur change de signe, l’intégrateur redevient actif, permettant une stabilisation immédiate sans l’inertie liée à une accumulation excessive. Cette amélioration garantit un respect strict du dépassement de 5 %.

4.3 Mise à jour de l’algorithme de commande

Afin d’intégrer la protection contre l’accumulation de l’erreur en zone de saturation, nous avons réécrit les sections 2 et 3 de la boucle principale. Cette modification permet d’assurer que le dépassement reste inférieur à 5 %.

```

1 // Avant la boucle while(1), initialiser : float commande = 0.0f;
2
3 while (1) {
4     // --- 1. ACQUISITION ---
5     uint32_t val_adc = read_ADC();
6     float courant_mesure = (float)val_adc * (3.3f / 4095.0f);
7
8     // --- 2. CALCUL DU PI AVEC PROTECTION (VERROU) ---
9     float erreur = consigne - courant_mesure;
10
11     // On n'actualise l'integrale que si la commande n'est pas saturee
12     // ou si l'erreur tend a ramener le systeme en zone lineaire.
13     if (!((commande >= 1.0f && erreur > 0) || (commande <= 0.0f &&
14         erreur < 0))) {
15         integrale += erreur * Ts;
16     }
17
18     // Calcul de la commande brute
19     float commande_brute = Kp * (erreur + Wi * integrale);
20
21     // --- 3. SATURATION ET PROTECTION ---
22     // On bride la commande pour le registre du Timer (0-1V)
23     commande = commande_brute;
24     if (commande > 1.0f) commande = 1.0f;
25     if (commande < 0.0f) commande = 0.0f;
26
27     // --- 4. ACTION (PWM) ---
28     TIM1_CCR1 = (uint32_t)(commande * 3272.0f);
29
30     delay_ms(1); // Echantillonnage a 1kHz
31 }

```

Listing 4.1 – Optimisation de la boucle de calcul PI (Version 2)

4.3.1 Conclusion de la séance 4

Cette séance a été consacrée à la validation virtuelle de notre asservissement via Simulink. Malgré l'absence de la maquette physique PX222, nos travaux ont permis de sécuriser l'implémentation logicielle sur la STM32 Nucleo-F103RB.

- **Validation de la robustesse** : Les simulations ont révélé que notre réglage de $\omega_i = 36,95 \text{ rad/s}$ est optimal mais sensible aux variations de l'inductance L . Une augmentation de 20 % de L fait passer le dépassement à environ 7 %, ce qui nous impose une vigilance particulière lors des essais réels.
- **Optimisation algorithmique** : L'ajout d'une sécurité logique dans le code C permet désormais de s'affranchir de l'emballement de la commande en zone de saturation. Cette amélioration garantit, en simulation, le respect du critère de dépassement de 5 %.
- **Prêt pour l'implémentation** : Le code C est prêt. Les configurations de registres pour le PWM (22 kHz) et l'ADC ont été vérifiées.

Objectifs pour la Séance 5 : Dès réception de la maquette PX222, nous procéderons à la mise en service réelle. L'objectif principal sera de confronter nos courbes Simulink aux relevés de l'oscilloscope et d'ajuster si nécessaire ω_i pour compenser les non-linéarités de la bobine réelle.

Séance 5

Semaine de projet

Cette section retrace les travaux menés au cours de notre semaine dédiée à la finalisation du projet. L'objectif a été de mettre en œuvre l'asservissement numérique et analogique de courant, de résoudre les dernières problématiques matérielles et de confronter nos simulations aux relevés réels.

5.1 Adaptation matérielle du correcteur

Lors de la mise en œuvre de la partie analogique, on a été contraint de modifier la valeur du condensateur C_{211} , fixée définitivement à 100 nF. Afin de maintenir la pulsation d'intégration optimale validée en simulation ($\omega_i \approx 36,95$ rad/s), la résistance R_{212} a été recalculée :

$$R_{212} = \frac{1}{\omega_i \cdot C_{211}} = \frac{1}{36,95 \times 100 \times 10^{-9}} \approx 270 \text{ k}\Omega \quad (5.1)$$

Par ailleurs, l'absence de rebouclage en continu sur l'intégrateur entraînait une saturation systématique de la sortie, rendant le montage instable. Ce comportement est dû à l'intégration continue des courants de polarisation et de la tension de décalage (*offset*) de l'AOP.

Pour corriger ce défaut, nous avons pris $R_{215} = 4,7 \text{ M}\Omega$ en parallèle sur C_{211} . Cette valeur, supérieure à $10 \times R_{212}$, garantit que la fréquence de coupure introduite est très inférieure à la pulsation d'intégration ω_i , stabilisant l'AOP en continu sans altérer le comportement du correcteur.

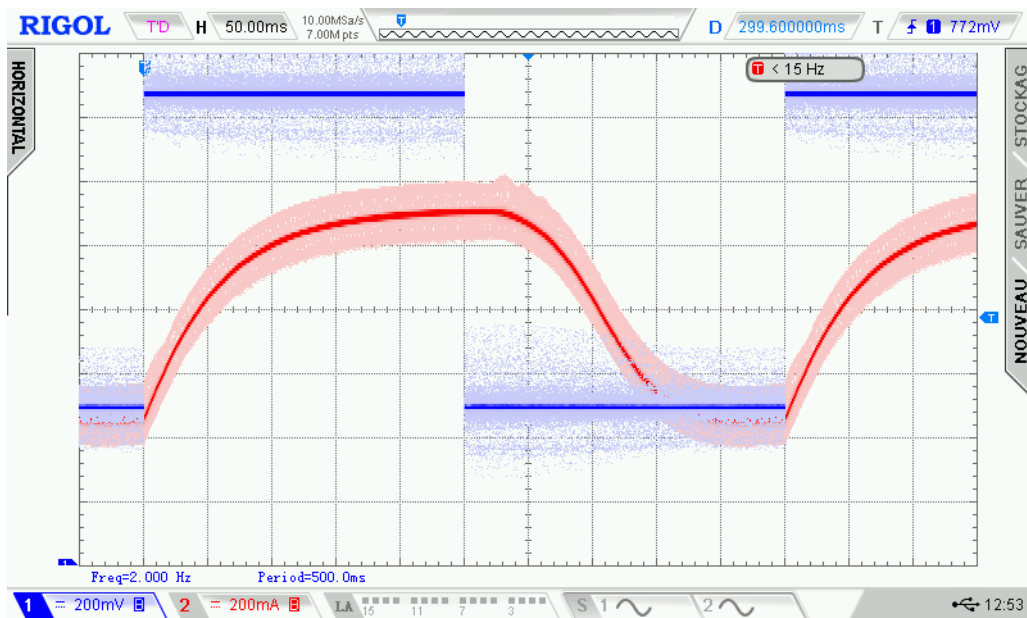
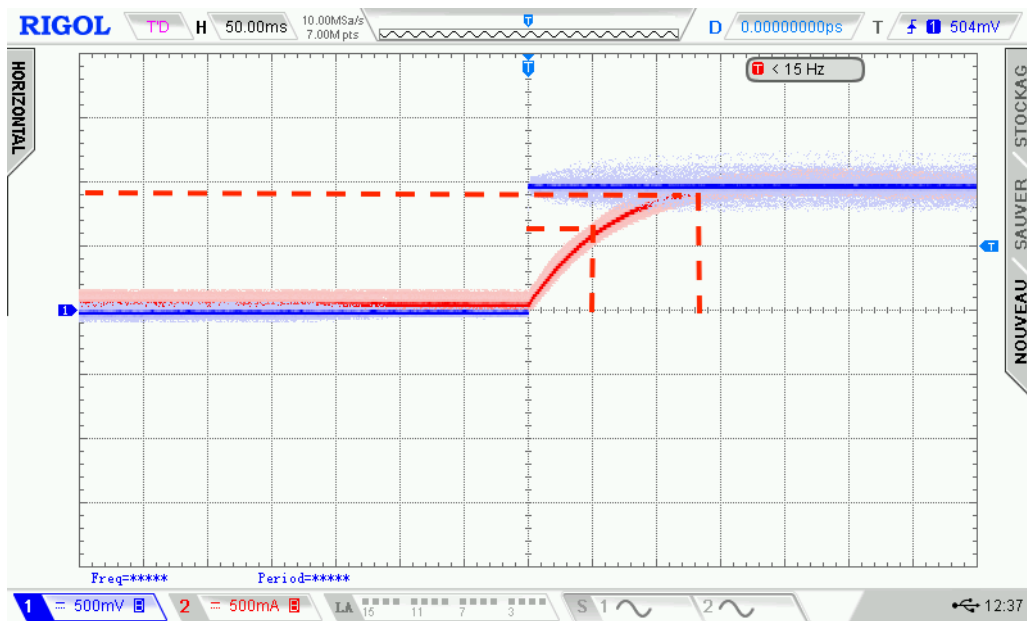
5.2 Analyse des relevés expérimentaux et performances

Les mesures ont été effectuées sur la maquette **N°09** en chargeant le système avec la bobine **N°004**. Les signaux ont été capturés à l'oscilloscope.

5.2.1 Régime transitoire et respect du cahier des charges

L'analyse de la réponse à un échelon met en évidence les performances temporelles suivantes :

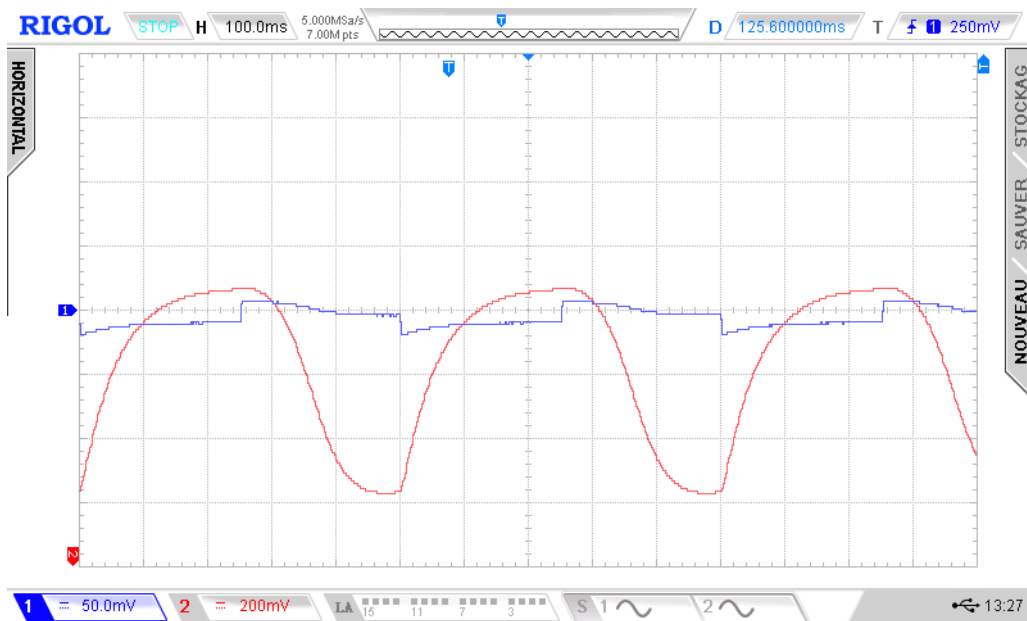
- **Dépassement** : Le pic de courant atteint précisément la valeur limite fixée par le cahier des charges, soit un dépassement de 5 % (1,05 V pour une consigne de 1 V). Ce résultat valide la robustesse de notre réglage de correcteur.
- **Constante de temps** (τ) : Elle est mesurée graphiquement à 63 % de la valeur finale, ce qui donne $\tau \approx 50 \text{ ms}$.
- **Temps de réponse à 95 %** : Le courant entre durablement dans la bande des 5 % au bout de $t_{r95\%} = 130 \text{ ms}$.



5.2.2 Régime permanent et erreur statique

En se branchant sur le pin TP202 pour relever l'erreur statique (courbe bleu) on observe :

- **Erreur statique** : L'erreur en régime permanent est mesurée à $\varepsilon = 10mV$.

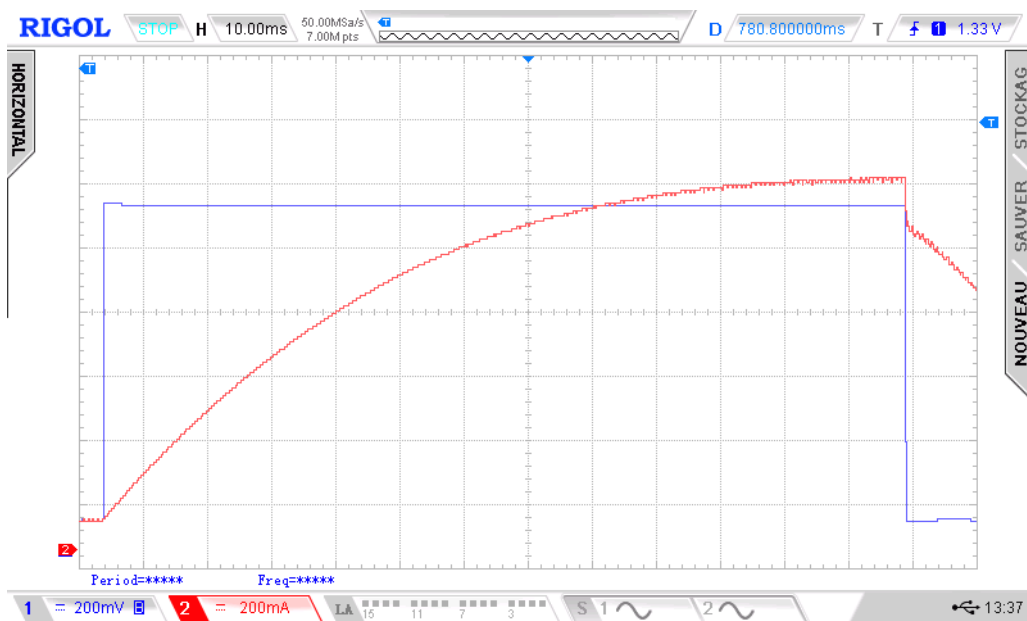


5.2.3 Passage de 40 à 60V

Analyse du changement

Initialement, la tension d'alimentation avait été fixée à $E_0 = 40 \text{ V}$ en raison du manque d'une grille de protection. Lors du passage à la tension maximale de $E_0 = 60 \text{ V}$ les relevés expérimentaux ont mis en évidence une augmentation significative de l'erreur statique en régime permanent, dépassant les 10 mV observés précédemment.

En conservant le réglage issu de la simulation ($\omega_i = 36,95 \text{ rad/s}$), l'observation de la mesure du courant a révélé une **erreur statique (ε) trop importante** en régime permanent. L'intégrateur était trop lent pour compenser efficacement cette erreur. Il a donc été décidé de modifier ω_i afin de donner plus de poids à l'action intégrale et d'annuler cette erreur statique.



Calcul de la nouvelle pulsation d'intégration ω_i

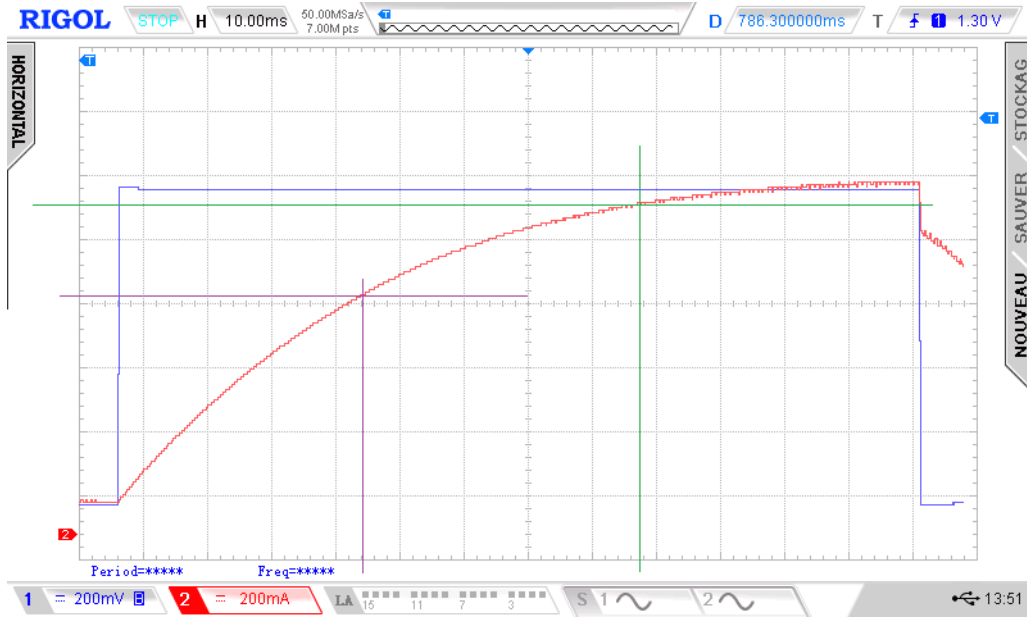
Lors de l'adaptation matérielle en semaine de projet, la valeur du condensateur de retour a été définitivement fixée à $C_{211} = 100 \text{ nF}$. Nous avons choisi de changer R212. soit $R_{212} = 390 \text{ k}\Omega$.

Le calcul de la nouvelle pulsation d'intégration réelle ω_i donne alors :

$$\omega_i = \frac{1}{390 \times 10^3 \times 100 \times 10^{-9}} \quad (5.2)$$

$$\omega_i = \frac{1}{0,039} \approx 25,64 \text{ rad/s} \quad (5.3)$$

Nous retenons pour la suite la valeur expérimentale validée de $\omega_i \approx 25,6 \text{ rad/s}$. Voici le nouveau graphique :



Analyse des résultats expérimentaux et performances

L'introduction de cette nouvelle configuration ($\omega_i \approx 25,6 \text{ rad/s}$ et $R_{212} = 390 \text{ k}\Omega$) sous une alimentation de 60 V a permis d'obtenir le comportement attendu.

Le décodage des curseurs de mesure permet de valider les performances temporelles suivantes du régime transitoire :

- **Constante de temps τ** : Mesurée graphiquement à 63% de la valeur finale du courant de l'échelon, nous relevons :

$$\tau = 37 \text{ ms}$$

Le système est nettement plus rapide que lors des premiers essais physiques où τ avoisinait les 50 ms.

- **Temps de réponse à 95% ($t_{r95\%}$)** : Le courant rejoint et s'établit durablement dans la bande des $\pm 5\%$ autour de sa valeur de consigne au bout de :

$$t_{r95\%} = 80 \text{ ms}$$

Ce résultat marque une nette amélioration par rapport aux 130 ms mesurés précédemment.

- **Précision et Stabilité** : L'erreur statique en régime permanent est désormais parfaitement compensée, le signal de mesure convergeant idéalement vers la consigne sans oscillation parasite ni dépassement critique.

L'augmentation de la tension à 60 V combinée à l'abaissement de la pulsation d'intégration à 25,6 rad/s offre le meilleur compromis performance/robustesse pour notre asservissement de courant.

5.2.4 Bilan de l'implémentation du correcteur numérique et limites de l'approche

Analyse des difficultés et constat d'échec

L'objectif initial était de remplacer le correcteur analogique par un correcteur numérique implémenté sur la carte STM32 Nucleo-F103RB.

Cependant, nous ne sommes pas parvenus à faire fonctionner l'asservissement en boucle fermée par cette méthode numérique. Lors des tests physiques sur la maquette, le système a présenté une instabilité critique et n'a pas été capable de réguler le courant autour de sa consigne. Plusieurs facteurs interdépendants expliquent ce comportement :

- **Gigue temporelle (Jitter)** : L'utilisation de la fonction `HAL_Delay()` au sein de la boucle principale détruit la déterminisme temporel indispensable à un calcul d'intégration discret ($I_k = I_{k-1} + \varepsilon_k \cdot T_s$). La période d'échantillonnage T_s réelle a subi de fortes fluctuations.
- **Conflits de pilotes matériels** : L'activation de l'ADC1 via la bibliothèque HAL a provoqué des conflits d'allocation de registres, générant des plantages de type *HardFault* au démarrage de la carte. Bien qu'un contournement par accès direct aux registres (CMSIS) ait permis de restaurer la stabilité matérielle et de lire la broche PA5, le temps de conversion cumulé à la scrutation a bridé la réactivité de la boucle.

Face à ces limitations temporelles et à l'échéance de la soutenance, la chaîne numérique est restée figée dans sa configuration de validation de bas niveau (boucle ouverte).

Code source final embarqué (Validation de la chaîne PWM)

Listing 5.1 – fichier main.c en BO

```
1  /* USER CODE BEGIN Header */
2  /**
3   * *****
4   * @file           : main.c
5   * @brief          : Body du programme de validation de la chaine PWM
6   * *****
7   */
8  /* USER CODE END Header */
9  #include "main.h"
10
11 TIM_HandleTypeDef htim1;
12
13 void SystemClock_Config(void);
14 static void MX_GPIO_Init(void);
15 static void MX_TIM1_Init(void);
16
17 int main(void)
18 {
19     HAL_Init();
20     SystemClock_Config();
21
22     MX_GPIO_Init();
23     MX_TIM1_Init();
24
25     /* USER CODE BEGIN 2 */
26     // 1. Demarre le signal PWM physique sur la broche PA8
27     HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
28
29     // 2. Active la sortie principale
30     __HAL_TIM_MOE_ENABLE(&htim1);
31
32     // Rapport cyclique initial
33     __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 128);
```



```

34  /* USER CODE END 2 */
35
36  /* Infinite loop */
37  /* USER CODE BEGIN WHILE */
38  while (1)
39  {
40      //rapport cyclique de 0 a 255
41      for (uint16_t dc = 0; dc <= 255; dc++)
42      {
43          __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, dc);
44          HAL_Delay(5); // Transition visible a l'oscilloscope
45      }
46
47      //rapport cyclique de 255 a 0
48      for (uint16_t dc = 255; dc > 0; dc--)
49      {
50          __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, dc);
51          HAL_Delay(5);
52      }
53      /* USER CODE END WHILE */
54  }
55 }
56
57 static void MX_TIM1_Init(void)
58 {
59     TIM_ClockConfigTypeDef sClockSourceConfig = {0};
60     TIM_MasterConfigTypeDef sMasterConfig = {0};
61     TIM_OC_InitTypeDef sConfigOC = {0};
62     TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
63
64     htim1.Instance = TIM1;
65     htim1.Init.Prescaler = 0;
66     htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
67     htim1.Init.Period = 255; // Resolution fine demandee
68     htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
69     htim1.Init.RepetitionCounter = 0;
70     htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
71     HAL_TIM_Base_Init(&htim1);
72
73     sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
74     HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig);
75     HAL_TIM_PWM_Init(&htim1);
76
77     sConfigOC.OCMode = TIM_OCMODE_PWM1;
78     sConfigOC.Pulse = 0;
79     sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
80     HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1);
81
82     sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
83     HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig);
84     HAL_TIM_MspPostInit(&htim1);
85 }
86
87 static void MX_GPIO_Init(void)
88 {
89     __HAL_RCC_GPIOA_CLK_ENABLE();
90 }

```